

Contents

Detecção Distribuída de Comunidades com Louvain	1
Documentação do Projeto — Código, Arquitetura e Uso Local	1
Sumário	1
1. Visão geral	2
2. Objetivos do trabalho	2
3. O algoritmo de Louvain	2
4. Dataset soc-pokec	3
5. Arquitetura do software	4
6. Estrutura de diretórios	4
7. Módulos e responsabilidades	5
8. Pré-processamento	6
9. Implementação Ray	7
10. Implementação Spark	7
11. Benchmark e relatórios	8
12. Interface de linha de comando	8
13. Testes automatizados	8
14. Como executar no seu PC (10%)	9
15. Decisões de engenharia	9
16. Referências bibliográficas	10

Detecção Distribuída de Comunidades com Louvain

Documentação do Projeto — Código, Arquitetura e Uso Local

Projeto: big-data / 001-distributed-louvain

Data: 26 de maio de 2026

Dataset: soc-pokec (SNAP)

Abordagens: Ray (manual) vs Spark + GraphFrames (Apostol et al., 2024)

Sumário

1. Visão geral
2. Objetivos do trabalho
3. O algoritmo de Louvain
4. Dataset soc-pokec
5. Arquitetura do software
6. Estrutura de diretórios
7. Módulos e responsabilidades
8. Pré-processamento
9. Implementação Ray
10. Implementação Spark
11. Benchmark e relatórios
12. Interface de linha de comando
13. Testes automatizados

14. Como executar no seu PC (10%)
15. Decisões de engenharia
16. Referências bibliográficas

1. Visão geral

Este projeto implementa e compara **duas variantes distribuídas** do algoritmo de **Louvain** para detecção de comunidades em grafos grandes:

Abordagem	Tecnologia	Descrição
Ray	Python + Ray Core	Louvain implementado manualmente com tasks remotas e batches de nós
Spark	PySpark + GraphFrames	Louvain via SQL/joins conforme Apostol et al. (2024)

Ambas leem o **mesmo artefato Parquet** (arestas normalizadas) gerado por um único pipeline de pré-processamento, garantindo comparação justa.

2. Objetivos do trabalho

1. Detectar comunidades na rede social eslovaca **soc-pokec** (~1,6M nós, ~30M arestas).
2. Comparar **desempenho** (tempo, memória, throughput) e **qualidade** (modularidade Q, número de comunidades).
3. Documentar trade-offs de **engenharia** (linhas de código, complexidade, infraestrutura).
4. Fornecer **testes unitários** para fórmulas centrais (ΔQ , modularidade, pré-processamento).

3. O algoritmo de Louvain

Referência: Blondel et al. (2008). O Louvain opera em dois estágios repetidos hierarquicamente:

Fase 1 — Otimização local

- Cada nó começa na própria comunidade.
- Para cada nó i , calcula-se o ganho ΔQ ao movê-lo para cada comunidade vizinha C :

$$\Delta Q = (k_{i_in} / m) - (\sigma_{tot} \times k_i) / (2m^2)$$

onde k_{i_in} = peso das arestas de i para C , σ_{tot} = soma dos graus em C , k_i = grau de i , m = soma total dos pesos / 2.

- O nó move-se para a comunidade com maior ($\Delta Q > 0$). Repete até estabilizar.

Fase 2 — Compressão

- Cada comunidade vira um **super-nó**; arestas entre comunidades agregam pesos.
- Reinicia Fase 1 no grafo comprimido.

Critério de parada (neste projeto)

Parar quando o ganho de modularidade **entre níveis hierárquicos** for menor que ϵ (padrão **1e-6**).

Modularidade Q

$$Q = (1/2m) \times \sum_{ij} (A_{ij} - (k_i \times k_j)/(2m)) \times \delta(c_i, c_j)$$

Valores típicos em redes sociais: 0,6–0,85. Acima de 0,3 indica estrutura comunitária relevante.

4. Dataset soc-pokec

Atributo	Valor
Fonte	SNAP — soc-Pokec
Arquivo	soc-pokec-relationships.txt
Nós (completo)	~1.632.803
Arestas (completo)	~30.622.564
Tipo	Direcionado no arquivo → tratado como não direcionado

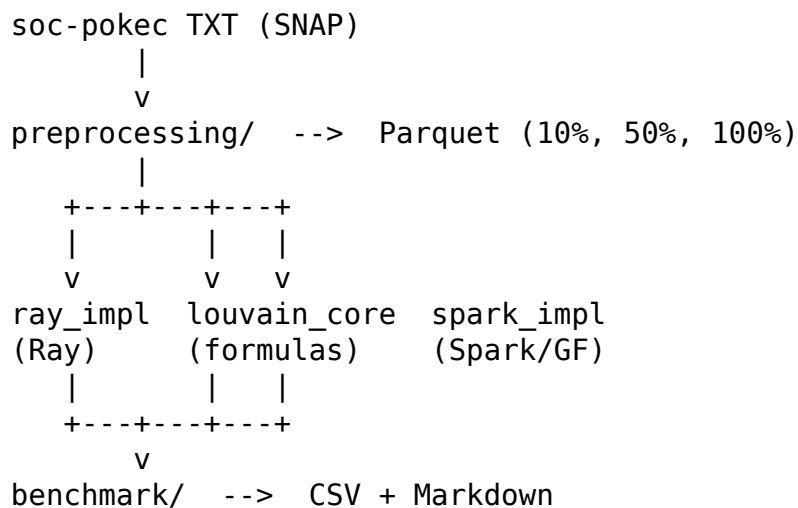
Pré-processamento aplicado

1. Remover cabeçalho e linhas inválidas.
2. Ignorar direção: $(u,v) = (v,u)$, manter $src < dst$.
3. Remover self-loops.
4. Peso uniforme **1.0**.
5. Subconjuntos experimentais: amostra **10%, 50%, 100%** dos nós (seed **42**).
6. Subgrafo **induzido** + maior componente conexa (**LCC**).
7. Exportar **Parquet** compartilhado (src, dst, weight).

Artefatos gerados (exemplo real)

Fração	Nós	Arestas	Ficheiro
10%	100.590	219.221	data/artifacts/pokec_10pct.parquet
50%	748.291	5.576.858	data/artifacts/pokec_50pct.parquet
100%	1.632.803	22.301.964	data/artifacts/pokec_100pct.parquet

5. Arquitetura do software



Princípios:

- **Modularidade:** Ray e Spark não importam um ao outro.
- **Código mínimo:** fórmulas Louvain centralizadas em `louvain_core/`.
- **Legibilidade:** um módulo por responsabilidade.

6. Estrutura de diretórios

```
big-data/
├── src/
│   ├── louvain_core/      # ΔQ, Q, compressão, Louvain sequencial
│   ├── preprocessing/    # SNAP → Parquet
│   ├── ray_impl/         # Louvain distribuído Ray
│   ├── spark_impl/       # Louvain GraphFrames
│   ├── benchmark/       # Métricas e relatório
│   ├── cli/             # Comandos main.py
│   └── config.py         # Configuração YAML/env
├── tests/
│   ├── unit/            # pytest
│   └── integration/
└──
```

```

├── data/
│   ├── raw/           # TXT SNAP
│   └── artifacts/     # Parquet por fração
├── reports/           # CSV + comparison.md
├── specs/001-distributed-louvain/ # Spec, plan, tasks
├── docs/              # Esta documentação
├── pyproject.toml
├── config.yaml.example
└── README.md

```

7. Módulos e responsabilidades

louvain_core/

Ficheiro	Função
graph.py	Grafo como lista de adjacência; propriedade m (peso total / 2)
delta_q.py	Cálculo de ΔQ e melhor movimento por nó
modularity.py	Modularidade Q
compress.py	Fase 2: agregar comunidades em super-nós
runner.py	Louvain sequencial completo (referência e testes)

preprocessing/

Ficheiro	Função
load_snap.py	Leitura streaming; SQLite em disco; export Parquet em batches
sample_lcc.py	Amostragem de nós, subgrafo induzido, LCC (NetworkX)
write_artifact.py	Escrita/leitura Parquet + .meta.json
validate_artifact.py	Validação de schema
pipeline.py	Orquestração preprocess

ray_impl/louvain_ray.py

- @ray.remote calcular_ganho_batch: avalia movimentos para um batch de nós.
- **Snapshot síncrono:** todos os workers usam a mesma partição; movimentos aplicados ao fim da rodada.
- ray.put implícito via passagem de adj e degree (otimizável com Object Store explícito).
- Mede init_time_s (ray.init) separado de algorithm_time_s.

spark_impl/

Ficheiro	Função
session.py	SparkSession + pacote GraphFrames
graph_loader.py	Parquet → GraphFrame
louvain_spark.py	Joins/groupBy para ΔQ ; checkpoint a cada 2 níveis

benchmark/

Ficheiro	Função
runner.py	Campanha $3 \times \text{runs} \times 2 \text{ abordagens} \times 3 \text{ frações} \rightarrow \text{CSV}$
metrics.py	tracemalloc (Ray)
report.py	Markdown comparativo (média $\pm$ desvio)
report_sections.py	Texto sobre race conditions + referências

cli/main.py

Subcomandos: preprocess, louvain-ray, louvain-spark, benchmark, report.

8. Pré-processamento

Fluxo

1. **Passagem 1:** percorrer o TXT e coletar IDs de nós únicos.
2. **Por fração (10/50/100):**
 - Amostrar `fraction_pct%` dos nós com `random.Random(seed)`.
 - **Passagem 2:** inserir arestas (ambos endpoints no conjunto) em SQLite temporário.
 - Se arestas $\leq 8\text{M}$: LCC em memória (NetworkX) → Parquet.
 - Se arestas $> 8\text{M}$ (100%): streaming SQLite → Parquet (LCC $\approx$ grafo completo no SNAP).

Comando

```
python -m src.cli.main preprocess \
  --input data/raw/soc-pokec-relationships.txt \
  --output-dir data/artifacts \
  --seed 42 \
  --fractions 10,50,100
```

Schema Parquet

Coluna	Tipo	Regra
src	int64	menor endpoint
dst	int64	maior endpoint
weight	float64	1.0

9. Implementação Ray

Paralelização (Fase 1)

1. Dividir nós em batches (ex.: 500–2000).
2. Lançar `calcular_ganho_batch.remote(...)` para cada batch.
3. `ray.get(refs)` em lote (nunca um a um em loop).
4. Aplicar todos os movimentos ao fim da rodada.
5. Repetir até nenhum movimento.

Race conditions (documentado)

Workers partilham o **mesmo snapshot** da partição. Conflitos no mesmo nó são esperados; o último movimento aplicado na rodada prevalece. O Louvain distribuído assim mesmo converge na prática.

Comando local (10%)

```
python -m src.cli.main louvain-ray \
  --artifact data/artifacts/pokec_10pct.parquet \
  --batch-size 2000 \
  --num-cpus 4
```

Dashboard: <http://localhost:8265>

10. Implementação Spark

Segue Apostol et al. (2024): Louvain **não é nativo** no GraphFrames; constrói-se com `join`, `groupBy`, `max_by` sobre DataFrames de arestas e comunidades.

Operações centrais

- `g.degrees` — graus dos nós.
- Join arestas $\times$ comunidades do destino $\rightarrow k_i_in$.
- Agregação `sigma_tot` por comunidade.
- Cálculo de `delta_q` e melhor comunidade por `src`.
- **Checkpoint** em vértices a cada 2 níveis hierárquicos (evita StackOverflow).

Comando local (10%)

```
export PYSPARK_SUBMIT_ARGS="--packages graphframes:graphframes:0.8.3-spark3.5-s_2.12"
export SPARK_CHECKPOINT_DIR=/tmp/spark_checkpoints
mkdir -p "$SPARK_CHECKPOINT_DIR"
```

```
python -m src.cli.main louvain-spark \
  --artifact data/artifacts/pokec_10pct.parquet
```

Spark UI: <http://localhost:4040>

Nota: na primeira execução o Maven baixa o JAR do GraphFrames (pode demorar).

11. Benchmark e relatórios

CSV (reports/metrics_raw.csv)

Colunas: approach, fraction_pct, run_index, tempos, memória, throughput, modularity_q, num_communities, epsilon, seed, converged, status, level_times_json.

Markdown (reports/comparison.md)

Tabelas média $\pm$ desvio padrão; secções Desempenho, Qualidade, Engenharia; discussão de race conditions Ray.

```
python -m src.cli.main benchmark --runs 3
python -m src.cli.main report
```

12. Interface de linha de comando

Comando	Descrição
preprocess	TXT SNAP → Parquet 10/50/100%
louvain-ray	Executa Louvain Ray num artefato
louvain-spark	Executa Louvain Spark num artefato
benchmark	Campanha completa de medições
report	Gera Markdown a partir do CSV

Variáveis de ambiente: POKEC_RAW_PATH, ARTIFACT_DIR, SEED, EPSILON, RAY_NUM_CPUS.

13. Testes automatizados

```
pytest tests/unit/ -v
pytest tests/integration/ -v
```


Teste	Cobre
test_modularity.py	Cálculo de Q
test_delta_q.py	Melhor movimento ΔQ
test_compress.py	Agregação Fase 2
test_preprocess.py	Normalização, LCC, Parquet
test_stop_epsilon.py	Parada por ϵ
test_edge_cases.py	Grafos vazios / isolados

14. Como executar no seu PC (10%)

Pré-requisitos

```
cd /caminho/para/big-data
python -m venv .venv && source .venv/bin/activate
pip install -e ".[dev]"
```

RAM orientativa

Algoritmo	10% (~100k nós)
Ray	4-8 GB
Spark	8-12 GB

Passo a passo

1. Confirmar artefato: `ls data/artifacts/pokec_10pct.parquet`
2. Rodar Ray (minutos a ~30 min conforme CPU).
3. Rodar Spark **depois** (não em paralelo com Ray).
4. Comparar JSON de saída (Q, tempos, comunidades).

15. Decisões de engenharia

Decisão	Motivo
Python vs Java (constituição original)	Ray/Spark/Python exigidos pelo trabalho académico
Parquet compartilhado	Mesmo input para Ray e Spark
SQLite no pré-processamento	Evitar OOM com 30M arestas
louvain_core compartilhado	Evitar duplicar fórmulas
LCC após amostragem	Reduzir ruído de nós isolados
$\epsilon = 1e-6$	Estabilidade numérica entre níveis

Decisão	Motivo
Streaming no 100%	Grafo completo não cabe em RAM para NetworkX

16. Referências bibliográficas

1. Blondel, V. D. et al. (2008). Fast unfolding of communities in large networks. *J. Stat. Mech.* P10008.
2. Apostol, E.-S., Cojocaru, A.-C., Truică, C.-O. (2024). Large-Scale Graphs Community Detection using Spark GraphFrames. IEEE ISPD. arXiv:2408.03966
3. Leskovec, J., Krevl, A. (2014). SNAP Datasets. <https://snap.stanford.edu/data>
4. Moritz, P. et al. (2018). Ray: A Distributed Framework for Emerging AI Applications. OSDI.
5. Dave, A. et al. (2016). GraphFrames: An Integrated API for Mixing Graph and Relational Queries.
6. Documentação Ray: <https://docs.ray.io/en/latest/ray-core/walkthrough.html>
7. GraphFrames: <https://github.com/graphframes/graphframes>

Documento gerado automaticamente a partir do código e especificações em specs/001-distributed-louvain/.